

report

Link to the source code: [GitHub Repository](#)

Authors

- Wojciech Bogacz 156034
- Krzysztof Skrobała 156039

Problem Description

The problem considered in this project involves a set of nodes, each represented by three integer values: two coordinates (x,y) that define the node's position in a two-dimensional plane, and a cost value associated with the node. The objective is to select exactly 50% of all available nodes (if the total number of nodes is odd, the number of selected nodes is rounded up) and construct a Hamiltonian cycle — that is, a closed path visiting each selected node exactly once and returning to the starting node. The goal is to minimize the sum of two components: 1. The total length of the constructed cycle, and 2. The total cost of all selected nodes. The distances between nodes are calculated using the Euclidean metric, rounded to the nearest integer.

Implemented Algorithms

```

FUNCTION MultistartLocalSearch(inst, num_starts, rnd) -> best_solution
    best_obj = +INF
    best_tour = nil
    best_inSel = nil

    FOR startAttempt FROM 0 TO num_starts-1:
        // create starting solution
        (tour0, inSel0) = RandomStart(inst, runRnd

        // apply local search (steepest in your current config)
        (tour_local, inSel_local, evals, imps) = RunLocalSearch(inst, tour0, inSel0, runRnd)

        // evaluate objective = tour length + selected costs
        tLen = TourLength(inst.Dist, tour_local)
        sCost = SelectedCosts(inst.Nodes, tour_local)
        obj = tLen + sCost

        IF obj < best_obj:
            best_obj = obj
            best_tour = copy(tour_local)
            best_inSel = copy(inSel_local)
        ENDIF
    ENDFOR

    RETURN (best_tour, best_inSel, best_obj)
END

```

```

FUNCTION ILS(inst, max_no_improve_iter, rnd) -> best_solution
    (tour_current, inSel_current) = RandomStart(inst, runRnd)
    (tour_current, inSel_current, _, _) = RunLocalSearch(inst, tour_current, inSel_current)

    best_tour = copy(tour_current)
    best_inSel = copy(inSel_current)

    best_obj = TourLength(inst.Dist, best_tour) + SelectedCosts(inst.Nodes, best_tour)

    REPEAT
    :
        // 1) Perturb current solution
        pertTour = copy(tour_current)
        pertInSel = copy(inSel_current)

        // (a) 5 random inter exchanges (replacing selected with unselected)
        FOR p FROM 1 TO 5:
            pos = runRnd.Intn(K)
            u = random unselected node (sample uniformly from [0..N-1] until !pertInSel[u])
            old = pertTour[pos]
            pertInSel[old] = false
            pertInSel[u] = true
            pertTour[pos] = u
        ENDFOR

        // (b) random intra changes
        FOR p FROM 1 TO 5:
            i = runRnd.Intn(K)
            j = runRnd.Intn(K)
            IF i == j:
                j = (i + 3) mod K
            ENDIF
            IF i > j: swap(i,j)
            reverse segment pertTour[i+1 .. j]
        ENDFOR

        (tour_new, inSel_new, evals, imps) = RunLocalSearch(inst, pertTour, pertInSel)

        // 3) Accept or reject
        new_obj = TourLength(inst.Dist, tour_new) + SelectedCosts(inst.Nodes, tour_new)
        prev_obj = TourLength(inst.Dist, tour_current) + SelectedCosts(inst.Nodes, tour_current)

        IF new_obj < prev_obj:
            tour_current = copy(tour_new)
            inSel_current = copy(inSel_new)
        ELSE
            best_tour = tour_current
            best_inSel = inSel_current
            best_obj = prev_obj
            BREAK
        ENDIF
    ENDWHILE
    RETURN (best_tour, best_inSel, best_obj)
END

```

Results

Comparison of previous methods results

Instance A

Method	Best	Worst	Average
random	235308	292123	264677.17
nn_end	89198	123123	104012.785
nn_anywhere	71488	74410	72635.72
greedy cycle	72639	72639	72639.0
2-Regret Insertion	105852	123428	115474.93
Weighted Sum ($\alpha=1.00$, $\beta=1.00$)	71108	73438	72130.85
greedy_intra:edges_start:greedy	70663	72648	71594.055
greedy_intra:edges_start:random	71564	76328	73528.81
greedy_intra:nodes_start:greedy	70687	73027	71739.395
greedy_intra:nodes_start:random	79677	99035	86909.13
steepest_intra:edges_start:greedy	70510	72614	71463.08
steepest_intra:edges_start:random	71246	78153	73699.995
steepest_intra:nodes_start:greedy	70626	73004	71615.93
steepest_intra:nodes_start:random	81322	95342	87939.335
steepest_intra:edges_start:random	71246	78153	73699.995
steepest_intra:edges_start:random with Cand mech.	73076	85487	77866.91
steepest_lm_intra:edges_start:random	71265	78247	73856.275

New Methods Results

Method	best	worst	average
ILS_intra:edges_start:random	70901	71862	71303.60
steepest_multi_start_intra:edges_start:random	71149	72794	71299.05

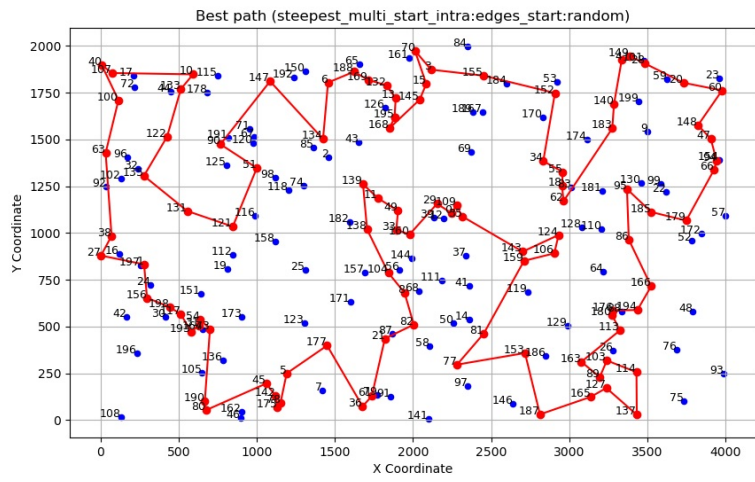
Instance B

Method	Best	Worst	Average
random	193234	234798	214279.235
nn_end	62606	77453	69764.43
nn_anywhere	49001	57324	51400.905
greedy cycle	50243	50243	50243.0
2-Regret Insertion	66505	77072	72454.77
Weighted Sum ($\alpha=1.00$, $\beta=1.00$)	47144	55700	50918.82
greedy_intra:edges_start:greedy	46178	55471	50172.67
greedy_intra:edges_start:random	45537	51687	48154.865
greedy_intra:nodes_start:greedy	46373	55385	50347.525
greedy_intra:nodes_start:random	53417	71474	61453.295
steepest_intra:edges_start:greedy	45867	54814	49907.205
steepest_intra:edges_start:random	45903	51416	48195.845
steepest_intra:nodes_start:greedy	46371	55385	50201.47
steepest_intra:nodes_start:random	55686	71546	62955.885
steepest_intra:edges_start:random	45903	51416	48195.845
steepest_intra:edges_start:random with Cand. Mech.	45798	52670	48474.64
steepest_lm_intra:edges_start:random	45941	51587	48214.715

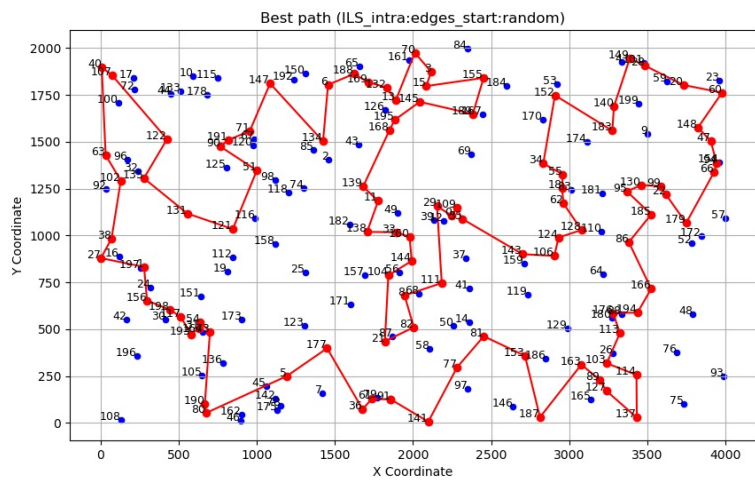
Method	best	worst	average
ILS_intra:edges_start:random	45320	46753	45463.3
steepest_multi_start_intra:edges_start:random	45405	46222	45767.4

Steepest with random starts





Iterated Local Search



Time

Instance A

Method	min	max	average
greedy_intra:edges_start:greedy	0.00	0.006	0.003
greedy_intra:edges_start:random	0.062	0.099	0.080
greedy_intra:nodes_start:greedy	0.000	0.005	0.002
greedy_intra:nodes_start:random	0.066	0.118	0.085
steepest_intra:edges_start:greedy	0.001	0.005	0.002
steepest_intra:nodes_start:greedy	0.000	0.005	0.002
steepest_intra:nodes_start:random	0.049	0.142	0.071
steepest_intra:edges_start:random	0.039	0.050	0.045
steepest_intra:edges_start:random with Can. Mech.	0.008	0.019	0.011
steepest_lm_intra:edges_start:random	0.025	0.049	0.030

New Methods Results

Method	min	max	average
--------	-----	-----	---------

IIS_intra:edges_start:random	4.805	4.815	4.811
steepest_multi_start_intra:edges_start:random	4.443	5.99	4.805

Instance B

Method	min	max	average
greedy_intra:edges_start:greedy	0.0	0.01	0.004
greedy_intra:edges_start:random	0.064	0.102	0.081
greedy_intra:nodes_start:greedy	0.0	0.007	0.003
greedy_intra:nodes_start:random	0.058	0.107	0.082
steepest_intra:edges_start:greedy	0.001	0.01	0.004
steepest_intra:nodes_start:greedy	0.001	0.007	0.004
steepest_intra:nodes_start:random	0.048	0.086	0.063
steepest_intra:edges_start:random	0.038	0.054	0.045
steepest_intra:edges_start:random with CAn. Mech.	0.009	0.05	0.012
steepest_lm_intra:edges_start:random	0.020	0.053	0.035

Method	min	max	average
IIS_intra:edges_start:random	4.499	4.515	4.503
steepest_multi_start_intra:edges_start:random	4.377	5.077	4.498

LS evaluations

Instance A

Method	min	max	average
IIS_intra:edges_start:random	410	7714	4064.5

Instance B

Method	min	max	average
IIS_intra:edges_start:random	344	7228	3786.1

Conclusions

The results demonstrate that the newly implemented methods, particularly IIS_intra:edges_start:random and steepest_multi_start_intra:edges_start:random, outperform many of the previously tested methods in terms of solution quality. For both instances A and B, these methods achieved lower average costs and competitive best-case results compared to other approaches.

The steepest_multi_start_intra:edges_start:random method, while computationally more expensive, consistently produced high-quality solutions, indicating its robustness and effectiveness for the problem at hand. On the other hand, IIS_intra:edges_start:random provided a good balance between computational efficiency and solution quality, making it a practical choice for scenarios with limited computational resources.

